

Chapitre 1 : Introduction à la programmation fonctionnelle et types de bases

I Introduction

Le paradigme de programmation détermine la manière dont un développeur conçoit et structure son code. On distingue principalement trois grandes familles :

- **Impératif** : Repose sur des instructions qui modifient l'état de la mémoire via des variables et des structures de contrôle (boucles, conditions). On décrit *comment* faire.
- **Orienté Objet** : Structure le code autour de l'interaction d'entités appelées objets, combinant données (attributs) et comportements (méthodes).
- **Fonctionnel** : Modélise le calcul comme l'évaluation de fonctions mathématiques, évitant le changement d'état et la mutation des données. On décrit *quoi* faire.

Remarque : La programmation fonctionnelle permet de limiter drastiquement les effets de bord (side-effects), rendant le code plus sûr, plus facile à tester et naturellement adapté au parallélisme.

Exemple : Écrivons un exemple simple de fonction selon les trois paradigmes : la fonction `map`, qui applique une fonction donnée à chaque élément d'une liste et retourne une nouvelle liste avec les résultats.

Impératif (Python) : On utilise une boucle explicite pour parcourir la liste.

```
1 def map_imperative(f, liste):
2     resultat = []
3     for element in liste:
4         resultat.append(f(element))
5     return resultat
```

Orienté Objet (Java) : L'opération est encapsulée dans une méthode.

```
1 public class Generateur {
2     public static <T, R> List<R> appliquerMap(Function<T, R> f, List<T> liste) {
3         return liste.stream().map(f).collect(Collectors.toList());
4     }
5 }
```

Fonctionnel (OCaml) : La fonction est définie par récursivité pure.

```
1 let rec map f liste =
2     match liste with
3     | [] -> []
4     | head :: rest -> (f head) :: (map f rest)
```

II Introduction à la programmation fonctionnelle

A Généralités sur OCaml

OCaml (Objective Caml) is un langage multi-paradigme, principalement fonctionnel, fortement et statiquement typé. Il utilise un mécanisme d'inférence de types automatique.

Définition : Un langage à **typage statique** vérifie la cohérence des types lors de la compilation, empêchant l'exécution d'un programme contenant des erreurs de type.

Définition : L'**inférence de types** est la capacité du compilateur à déterminer automatiquement le type d'une expression sans que le développeur ait besoin de le spécifier explicitement.

Le langage OCaml est particulièrement apprécié pour sa robustesse, sa performance et sa capacité à gérer des abstractions complexes tout en restant expressif.

B Types de base en OCaml

OCaml possède plusieurs types de données primitifs non mutables :

- `int` : Les entiers (ex: 42).
- `float` : Les nombres à virgule flottante (ex: 3.14).
- `bool` : Les booléens (`true` ou `false`).
- `char` : Les caractères entourés de simples quotes (ex: `'a'`).
- `string` : Les chaînes de caractères entourées de guillemets (ex: `"OCaml"`).

C Opérations sur les types de base

Chaque type de base en OCaml possède son propre ensemble d'opérateurs dédiés. Le compilateur étant strict sur le typage, il est impossible de mélanger des types différents au sein d'une même opération sans conversion explicite.

1 Les Entiers (`int`)

Les opérations arithmétiques usuelles sur les entiers utilisent les symboles standards. Le résultat de ces opérations est toujours un entier.

- Addition : `+`
- Soustraction : `-`
- Multiplication : `*`
- Division entière : `/` (tronque la partie décimale)
- Modulo (reste de la division) : `mod`

💡 **Exemple** : `7 / 2` renvoie 3, tandis que `7 mod 2` renvoie 1.

2 Les Flottants (`float`)

✖ **Attention** ✖ En OCaml, les opérateurs pour les nombres à virgule flottante sont distincts de ceux des entiers. Ils sont suivis d'un point (`.`).

Les opérateurs pour les flottants sont :

- Addition : `+.`
- Soustraction : `-.`
- Multiplication : `*.`
- Division : `/.`

✖ **Attention** ✖ Le typage statique d'OCaml interdit les opérations mixtes. L'expression `2 + 3.0` ou `2 +. 3.0` provoquera une erreur immédiate de compilation (*Type error*).

Méthode : Conversion de type (Cast)

Pour effectuer un calcul entre un entier et un flottant, il faut explicitement modifier l'un des deux types à l'aide des fonctions :

- `float_of_int` : convertit un `int` en `float`.
- `int_of_float` : convertit un `float` en `int` (en tronquant).

Exemple valide : `float_of_int 2 +. 3.0` renvoie `5.0`.

3 Les Booléens (`bool`)

Les expressions booléennes permettent de réaliser des tests logiques. Les opérateurs principaux sont :

- Et logique : `&&`
- Ou logique : `||`
- Négation : `not`

❗ **Remarque** : Les opérateurs de comparaison (`=`, `<>`, `<`, `>`, `<=`, `>=`) s'appliquent aux types de base et renvoient une valeur booléenne. Notez qu'en OCaml, l'égalité structurelle se teste avec un seul égal (`=`) et la différence avec `<>`.

4 Les Chaînes et Caractères (`string` et `char`)

L'opération principale sur les chaînes de caractères est la concaténation (la fusion de deux chaînes), qui utilise l'opérateur `^`.

💡 **Exemple** : L'expression `"Java" ^ "Script"` renvoie la chaîne `"JavaScript"`.

D Fonctions et expressions

1 Conditionnelles

Pour effectuer des tests conditionnels, OCaml utilise l'expression `if ... then ... else ....` Contrairement à certains langages, cette expression est obligatoire pour les deux branches (le `else` est nécessaire).

On peut comparer les valeurs avec les opérateurs de comparaison (`=`, `<>`, `<`, `>`, `<=`, `>=`) et combiner les conditions avec les opérateurs logiques (`&&` pour "et", `||` pour "ou").

✖ **Attention** ✖ En OCaml, c'est bien le `=` qui est utilisé pour tester l'égalité, et non le `==` comme dans certains autres langages.

2 Expressions

En programmation fonctionnelle, tout est expression et renvoie une valeur. Les fonctions sont des "citoyens de première classe" : elles peuvent être passées en argument ou renvoyées par d'autres fonctions.

3 Fonctions

Méthode : Déclaration d'une variable ou d'une fonction globale

On utilise le mot-clé `let` pour lier un nom à une valeur ou à une fonction de manière permanente au niveau global du fichier.

Syntaxe d'une variable globale : `let nom = expression`

Syntaxe d'une fonction globale : `let nom_fonction arg1 arg2 = corps`

 **Vocabulaire** : Une fonction est dite d'**ordre supérieur** si elle prend une ou plusieurs fonctions en argument et/ou renvoie une fonction en résultat.

 **Exemple** : Pour écrire une fonction qui calcule le carré d'un entier, on peut déclarer :

```
1 (* Declaration d'une fonction simple *)
2 let carre x = x * x
3
4 (* Utilisation de la fonction *)
5 let resultat = carre 5
```

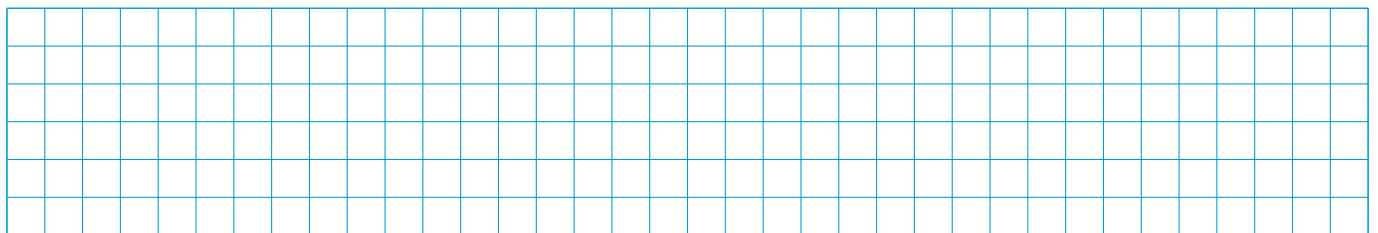
✗ Attention ✗ Par défaut, les fonctions en OCaml ne sont pas récursives. Pour qu'une fonction puisse s'appeler elle-même, il faut obligatoirement utiliser le mot-clé `let rec`.

 **Exemple** : Pour calculer le n -ième terme de la suite de Fibonacci, on peut déclarer :

```
1 let rec fibonacci n =
2   if n <= 1 then n
3   else fibonacci (n - 1) + fibonacci (n - 2)
```

Notons qu'il n'est pas nécessaire de mettre des parenthèses autour des arguments d'une fonction, sauf si l'on souhaite regrouper plusieurs arguments en une seule expression. Les parenthèses permettent toutefois de clarifier l'ordre d'évaluation et de gérer la priorité des opérations.

 **Application** : Écrire une fonction récursive factorielle : `int -> int` qui calcule la factorielle d'un entier n .



4 Liaisons locales : la structure `let ... in ...`

Définition : Une liaison locale permet de définir une variable ou une fonction intermédiaire dont la portée (visibilité) est restreinte exclusivement à une expression spécifique. On utilise pour cela le mot-clé `let ... in`

 **Remarque** : Contrairement à un `let` global, un `let ... in` est lui-même une **expression** et produit une valeur de retour (celle de la partie après le `in`). Elle permet d'éviter de polluer l'espace de nom global du programme et s'avère indispensable pour décomposer des calculs ou forcer un ordre d'évaluation lors d'effets de bord.

Méthode : Structure du `let ... in`

La syntaxe générale est la suivante :

$$\text{let } \textit{nom} = \textit{expression_locale} \text{ in } \textit{expression_principale}$$

Le `nom` n'est accessible et connu que dans l'*expression_principale*. Dès que celle-ci est évaluée, la variable locale est oubliée et détruite de la mémoire.

 **Exemple** : Exemple d'utilisation de liaisons locales pour calculer la racine d'un polynôme ou décomposer une formule géométrique :

```

1 (* Calcul du perimetre d'un rectangle avec des variables locales *)
2 let perimetre la lo =
3   let double_largeur = 2 * la in
4   let double_longueur = 2 * lo in
5   double_largeur + double_longueur
6
7 (* double_largeur n'existe plus ici, l'appeler provoquerait une erreur *)

```

5 Signature d'une fonction et unit

Définition : On appelle **unit** le type de la valeur renvoyée par une fonction qui ne retourne rien de significatif. Il est représenté par le mot-clé `unit` et sa seule valeur est `()`. Une fonction de type `unit` est souvent utilisée pour ses effets secondaires, comme l'affichage à l'écran ou la modification d'une variable globale.

Exemple : Le type `unit` est utilisé dans la fonction `print_newline : unit -> unit` qui affiche un saut de ligne à l'écran. Elle ne prend aucun argument et ne renvoie aucune valeur significative.

```

1 (* Exemple d'utilisation de print_newline *)
2 let afficher_message () =
3   print_string "Bonjour, monde !";
4   print_newline () (* Affiche un saut de ligne *)

```

Méthode : Signature d'une fonction

La signature d'une fonction décrit son type, c'est-à-dire le type de ses arguments et le type de sa valeur de retour. Elle est exprimée sous la forme :

$$nom_fonction : type_arg1 \rightarrow type_arg2 \rightarrow \dots \rightarrow type_resultat$$

Exemple : La fonction `carre` a pour signature `carre : int -> int`, indiquant qu'elle prend un entier en argument et renvoie un entier.

Exemple : La fonction `print_string` a pour signature `print_string : string -> unit`, ce qui signifie qu'elle prend une chaîne de caractères en argument et ne renvoie aucune valeur significative (le type `unit` est utilisé pour indiquer l'absence de valeur).

III Digression : utilisation du ";" et du ";;"

Comme on l'a dit, chaque expression en OCaml renvoie une valeur. Il convient de ne pas confondre le rôle du point-virgule simple `;` et du double point-virgule `;;`.

- **Le point-virgule simple (;) :** C'est un **séquenceur d'instructions**. Il s'utilise uniquement à l'intérieur d'un bloc de code pour enchaîner des expressions (généralement de type `unit`) possédant des effets de bord. L'expression de gauche est évaluée, son résultat est jeté, puis l'expression de droite est exécutée. La dernière expression d'un bloc ne prend jamais de point-virgule simple.
- **Le double point-virgule (;;) :** C'est un **délimiteur de phrase globale**. Il sert principalement dans la console interactive (le REPL, c'est-à-dire le mode interactif du compilateur) pour signifier au compilateur que la saisie est terminée et qu'il peut exécuter le bloc. Dans un fichier source (`.ml`), il est facultatif si les déclarations globales s'enchaînent par des mots-clés clairs (`let`), mais il reste utile pour exécuter directement des instructions libres au niveau global.

Exemple : Pour afficher deux messages consécutifs et clore la fonction dans le REPL, on combine les deux syntaxes :

```
1 let saluer () =  
2   print_string "Hello, "; (* Le ";" separe deux expressions unit *)  
3   print_string "world !"  (* Pas de ";" sur la derniere expression *);;  
4   (* Le ";;" indique a l'interpreteur que la definition est finie *)
```

Contents

I	Introduction	1
II	Introduction à la programmation fonctionnelle	1
A	Généralités sur OCaml	1
B	Types de base en OCaml	2
C	Opérations sur les types de base	2
1	Les Entiers (<code>int</code>)	2
2	Les Flottants (<code>float</code>)	2
3	Les Booléens (<code>bool</code>)	3
4	Les Chaînes et Caractères (<code>string</code> et <code>char</code>)	3
D	Fonctions et expressions	3
1	Conditionnelles	3
2	Expressions	3
3	Fonctions	3
4	Liaisons locales : la structure <code>let</code> ... <code>in</code>	4
5	Signature d'une fonction et <code>unit</code>	5
III	Digression : utilisation du ";" et du ";;"	5

Crédits

Ce cours est mis à disposition selon les termes de la Licence Creative Commons  **Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International**. Pour voir une copie de cette licence, visitez <http://creativecommons.org/licenses/by-nc-sa/4.0/>.